

TECHNICAL TASK

Project Engineering & Architecture Assignment: Grid Puzzle Challenge

1. Objective

The goal of this assignment is to design and develop a polished, playful, grid-based mobile puzzle game. This task is engineered to evaluate your core problem-solving logic, architectural engineering practices, adherence to data flow principles, and source control workflow. Your final application must blend a mathematically reliable logical backend with a clear and seamless user interface.

2. Core Game Mechanics & System Requirements

Your implementation must strictly fulfill the following deterministic requirements. All edge cases regarding spatial tracking and input validation should be cleanly handled.

- **$N \times M$ Grid Simulation:** Build a logical grid management system. The grid data structural model must remain entirely independent from the visual rendering layer.
- **Swipe-Based Input System:** Implement a robust gesture detection or pointer vector tracking pipeline supporting discrete directional commands: Up, Down, Left, and Right.
- **Deterministic State History (Undo/Moves):** Integrate a dedicated state tracking mechanic. The application must monitor every logical board transition, allowing players to either reverse history via a clear "Undo" action or solve the layout within a rigid step framework.
- **User Interface & Presentation:** Create a user interface optimized for clear comprehension. The layout must prominently convey current system metrics (e.g., score values or remaining action steps), feature explicit feedback during execution, and render clear game-over/win states.



Candidate Initiative: Hook & Mechanics

Beyond the core constraints, you are expected to design and execute an "Extra Feature" or "Gameplay Hook"—such as unique power-ups, synergistic combos, or specialized logical obstacles—to heighten player retention and show your creativity.

3. Mandatory Engineering Deliverables

Your submission will be evaluated as a production-ready codebase. The following deliverables must be present upon final evaluation:

3.1. Source Control & Build Delivery

- The project must be initialized as a public or accessible GitHub repository from inception.

- **Commit Discipline:** Code must be built incrementally using regular, atomic commits. Large, single-package drops at completion are strictly disqualifying. Ensure messages follow structured formatting conventions (e.g., semantic prefixes like `feat:`, `fix:`, `refactor:`). Type your text
- **Executable Build Submission:** Provide a stable, compiled `.apk` build link within the repository's main release page or via a direct download link inside the documentation for structural testing.

3.2. Technical Architecture & Data Documentation

You must embed precise architectural diagrams within your repository’s root directory or native `README.md` file. These diagrams must map out the structural design patterns utilized:

System Architecture Map	A comprehensive model showing concrete component relationships, strict separation of concerns, and decoupling between core state logic and UI rendering frameworks.
Functional Code Flow	A definitive lifecycle diagram charting the complete data pipeline from initial user input events: <i>User Gesture → Input Controller → Grid Matrix Update → UI Rendering Framework.</i>

4. Evaluation Rubric

Submissions will be programmatically and structurally analyzed using the criteria listed below:

1. **Code Architecture & Readability:** Decoupling of state data, adherence to single responsibility principles, object-oriented/functional clarity, and strict type safety.
2. **Algorithmic Efficiency:** Execution performance of grid update procedures and history tracking memory safety (avoiding deep-copy overhead or state-tracking leaks).
3. **Git History Integrity:** The cadence, granularity, and professional documentation visible across your commit history timelines.

Note: Task duration 4 days from assigned date.